

给 Agent 加上语音交互：ASR + 流式 TTS 已付费

原创 神说要有光 神光的幸福生活 2026年3月28日 11:08 山东

我们常用的 Agent 都有语音功能。

比如你用豆包的时候：

神光的编程秘籍

语音输入会转成文字，大模型的回答会通过语音朗读。可以切换音色。

这种 STT (Speech To Text) 语音转文字，TTS (Text To Speech) 文字转语音基本是 Agent 开发必备技术了。

这节我们就来学一下语音相关技术，实现豆包同款功能。

创建项目：

```
mkdir tts-stt-test
cd tts-stt-test
npm init -y
```

我们用腾讯云的语音（各家用法都差不多）。

<https://console.cloud.tencent.com/tts>

神光的编程秘籍

拿到 secretId、secretKey 之后，就可以调用 api 了。

创建 src/tts-test.mjs

```
import "dotenv/config";
import tencentcloud from "tencentcloud-sdk-nodejs-tts";
import fs from "node:fs";

const secretId = process.env.SECRET_ID;
const secretKey = process.env.SECRET_KEY;

const TtsClient = tencentcloud.tts.v20190823.Client;

const client = new TtsClient({
  credential: {
    secretId,
    secretKey,
  },
  region: "ap-beijing",
  profile: {
    httpProfile: {
      endpoint: "tts.tencentcloudapi.com",
    },
  },
});
```

```

const params = {
  Text: "下班路上，我还在为晚霞开心。突然电话响起：系统崩了。我的心一下揪紧，冲进办公室时几乎要绝望。可当大！
  SessionId: "session-001",
  VoiceType: 502006,           // 101007: 智瑜 (女声)
  Codec: "mp3",                // 指定输出格式为 mp3
};

client.TextToVoice(params).then(
  (data) => {
    // 返回的 Audio 字段是 Base64 编码的音频数据
    const audioBuffer = Buffer.from(data.Audio, "base64");
    const outputPath = "./output.mp3";

    fs.writeFile(outputPath, audioBuffer, (err) => {
      if (err) {
        console.error("保存文件失败: ", err);
      } else {
        console.log("MP3 已保存至: ", outputPath);
      }
    });
  },
  (err) => {
    console.error("合成失败: ", err);
  }
);

```

调用文字转语音 tts 功能，传入参数，返回的 base64 字符串转为 buffer 写入文件。

这个音色 id 从这里找：

<https://cloud.tencent.com/document/product/1073/92668>

安装用到的包：

```
pnpm install dotenv tencentcloud-sdk-nodejs-tts
```

创建 .env 配置文件：

```
SECRET_ID=替换成你的  
SECRET_KEY=替换成你的
```

跑一下：

但这种直接传入全部文本生成语音的方式，显然不太适合我们的场景。

比如豆包流式返回回答，语音也是流式播放的。

这种就需要用流式语音合成接口了，它是 websocket 的

<https://cloud.tencent.com/document/product/1073/108595>

创建 src/streaming-tts-test.mjs

```
import "dotenv/config";
import WebSocket from "ws";
import crypto from "node:crypto";
import fs from "node:fs";

const SECRET_ID = process.env.SECRET_ID;
const SECRET_KEY = process.env.SECRET_KEY;
const APP_ID = process.env.APP_ID;

const VOICE_TYPE = 101001;
const OUTPUT_FILE = "output3.mp3";
const TEXT_INTERVAL_MS = 3000;
const TEXTS = [
  "傍晚我还在为晚霞开心，",
  "突然接到电话说系统崩了，",
  "我心里一沉冲回办公室，",
  "好在大家一起排查后终于恢复，",
  "我长长松了口气。",
];

const sleep = (ms) => new Promise((resolve) => setTimeout(resolve, ms));
```

```

function buildWsUrl() {
  const now = Math.floor(Date.now() / 1000);
  const sessionId = `session_${now}_${Math.random().toString(36).slice(2)}`;

  const params = {
    Action: "TextToStreamAudioWSv2",
    AppId: parseInt(APP_ID),
    Codec: "mp3",
    Expired: now + 3600,
    SampleRate: 16000,
    SecretId: SECRET_ID,
    SessionId: sessionId,
    Speed: 0,
    Timestamp: now,
    VoiceType: VOICE_TYPE,
    Volume: 5,
  };

  const sortedKeys = Object.keys(params).sort();
  const signStr = sortedKeys.map((k) => `${k}=${params[k]}`).join("&");
  const rawStr = `GETtts.cloud.tencent.com/stream_wsv2?${signStr}`;
  const signature = crypto
    .createHmac("sha1", SECRET_KEY)
    .update(rawStr)
    .digest("base64");
  const searchParams = new URLSearchParams({
    ...params,
    Signature: signature,
  });

  return {
    sessionId,
    url: `wss://tts.cloud.tencent.com/stream_wsv2?${searchParams.toString()}`,
  };
}

asyncfunction sendTexts(ws, sessionId) {
  for (let i = 0; i < TEXTS.length; i++) {
    ws.send(JSON.stringify({ session_id: sessionId, message_id: `msg_${i}`, action: "ACTI
    console.log(`[文本] 已发送: ${TEXTS[i]}`);
    if (i < TEXTS.length - 1) await sleep(TEXT_INTERVAL_MS);
  }
  ws.send(JSON.stringify({ session_id: sessionId, action: "ACTION_COMPLETE" }));
  console.log("[文本] 已发送 ACTION_COMPLETE");
}

```

```
function streamTTS() {
  if (!SECRET_ID || !SECRET_KEY || !APP_ID) {
    throw new Error("请先在 .env 配置 SECRET_ID、SECRET_KEY、APP_ID");
  }

  const { url, sessionId } = buildWsUrl();
  const ws = new WebSocket(url);
  const writeStream = fs.createWriteStream(OUTPUT_FILE, { flags: "w" });
  let totalBytes = 0;
  let closed = false;
  let sent = false;

  const closeAll = () => {
    if (closed) return;
    closed = true;
    writeStream.end(() => {
      console.log(`[保存] 音频已保存至 ${OUTPUT_FILE}, 共 ${totalBytes} 字节`);
    });
    if (ws.readyState < WebSocket.CLOSING) ws.close();
  };

  ws.on("open", () => {
    console.log("[连接] WebSocket 已建立, 等待服务端就绪...");
  });

  ws.on("message", async (data, isBinary) => {
    if (isBinary) {
      writeStream.write(data);
      totalBytes += data.length;
      return;
    }

    try {
      const msg = JSON.parse(data.toString());
      console.log("[消息]", JSON.stringify(msg));

      if (msg.ready === 1 && !sent) {
        sent = true;
        await sendTexts(ws, sessionId);
      }

      if (msg.code && msg.code !== 0) {
        console.error(`[错误] code=${msg.code}, message=${msg.message}`);
        closeAll();
      } else if (msg.final === 1) {
        console.log("[完成] 合成结束。");
        closeAll();
      }
    }
  });
}
```

```

    } catch (e) {
      console.error("[解析错误]", e.message);
    }
  });

ws.on("error", (err) => {
  console.error("[WebSocket 错误]", err.message);
  closeAll();
});

ws.on("close", (code, reason) => {
  console.log(`[断开] 连接已关闭, code=${code}, reason=${reason}`);
  closeAll();
});
}

streamTTS();

```

我们先构造了 url，用 WebSocket 连上

每 3s 发送一次消息

然后用 fs.createWriteStream 异步写入文件

appid 从这里拿：

加到 .env 里：

跑一下：

神光的编程秘籍

因为文本是流式返回的，所以语音一般也要流式生成，用 streaming tts 的接口。

接下来试一下语音识别 ASR (Automatic Speech Recognition)，叫 STT (Speech To Text) 也可以，但 ASR 用的多一些。

这个就不用流式了。你平时用豆包的时候，都是说完一段话才转成的文本

创建 src/asr-test.mjs

```
import "dotenv/config";
import tencentcloud from "tencentcloud-sdk-nodejs";
import fs from "node:fs";

const SECRET_ID = process.env.SECRET_ID;
const SECRET_KEY = process.env.SECRET_KEY;

const AsrClient = tencentcloud.asr.v20190614.Client;
const AUDIO_FILE = './output.mp3';

const client = new AsrClient({
  credential: {
    secretId: SECRET_ID,
    secretKey: SECRET_KEY,
  },
  region: "ap-shanghai",
  profile: {
    httpProfile: {
```

```

    reqMethod: "POST",
    reqTimeout: 30,
  },
},
});

asyncfunction run() {
  const audioBase64 = fs.readFileSync(AUDIO_FILE).toString("base64");

  const params = {
    EngSerViceType: "16k_zh",
    SourceType: 1,
    Data: audioBase64,
    DataLen: Buffer.byteLength(audioBase64),
    VoiceFormat: "mp3",
  };

  try {
    const data = await client.SentenceRecognition(params);
    console.log("识别结果: ", data.Result);
  } catch (err) {
    console.error("识别失败: ", err);
  }
}

run();

```

传入音频 mp3 文件，调用接口来识别，返回文本

安装下依赖：

```
pnpm install tencentcloud-sdk-nodejs
```

跑一下：

这样，我们就可以来实现豆包同款的语音交互了：

点击录音，输入一段语音，服务端提供接口来转文字，之后用大模型生成回答。

流式 SSE 返回文字，同时用 WebSocket 返回流式语音。

这样就可以实现语音输入，流式的文字、语音输出。

为啥不直接用 SSE 返回音频数据呢？

因为 SSE 是基于 http 的文本协议，需要转 Base64 才行，传这种二进制数据还是 WebSocket 更合适。

思路理清了，接下来按照这个实现下豆包同款交互：

先创建后端项目：

```
nest new asr-and-tts-nest-service
```

先写一下调用大模型回答的 SSE 接口

创建 ai 模块：

```
nest g module ai
nest g controller ai --no-spec
nest g service ai --no-spec
```

改下 AiService：

```
import { Inject, Injectable } from '@nestjs/common';
import { ChatOpenAI } from '@langchain/openai';
import { PromptTemplate } from '@langchain/core/prompts';
import type { Runnable } from '@langchain/core/runnables';
import { StringOutputParser } from '@langchain/core/output_parsers';

@Injectable()
export class AiService {
  private readonly chain: Runnable;

  constructor(
    @Inject('CHAT_MODEL') model: ChatOpenAI
  ) {
    const prompt = PromptTemplate.fromTemplate(
      '请回答以下问题：\n\n{query}',
    );
    this.chain = prompt.pipe(model).pipe(new StringOutputParser());
  }

  async *streamChain(query: string): AsyncGenerator<string> {
    const stream = await this.chain.stream({ query });
    for await (const chunk of stream) {
      yield chunk;
    }
  }
}
```

还有 AiController：

```

import { Controller, Get, Query, Sse } from '@nestjs/common';
import { from, map, Observable } from 'rxjs';
import { AiService } from './ai.service';

@Controller('ai')
export class AiController {
  constructor(private readonly aiService: AiService) {}

  @Sse('chat/stream')
  chatStream(@Query('query') query: string): Observable<{ data: string }> {
    return from(this.aiService.streamChain(query)).pipe(
      map((chunk) => ({ data: chunk })),
    );
  }
}

```

和 AiModule:

```

import { Module } from '@nestjs/common';
import { AiService } from './ai.service';
import { AiController } from './ai.controller';
import { ConfigService } from '@nestjs/config';
import { ChatOpenAI } from '@langchain/openai';

@Module({
  controllers: [AiController],
  providers: [AiService,
    {
      provide: 'CHAT_MODEL',
      useFactory: (configService: ConfigService) => {
        return new ChatOpenAI({
          model: configService.get('MODEL_NAME'),
          apiKey: configService.get('OPENAI_API_KEY'),
          configuration: {
            baseUrl: configService.get('OPENAI_BASE_URL'),
          },
        });
      },
      inject: [ConfigService],
    }
  ],
})
export class AiModule {}

```

就是基于用 langchain 创建一个 chain 来回答用户的问题，流式返回

安装用到的包：

```
pnpm install @nestjs/config @langchain/openai @langchain/core
```

创建配置文件 .env

```
OPENAI_API_KEY=sk-xxx  
OPENAI_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1  
MODEL_NAME=qwen-plus
```

在 AppModule 里引入下：

这些前面写过，就是再熟悉一遍。

跑一下：

我们先接入语音转文字，实现一个接口：

创建 speech 模块：

```
nest g module speech
nest g service speech --no-spec
nest g controller speech --no-spec
```

把之前 asr 的逻辑拿过来，放到 service 里：

```
import { Inject, Injectable } from '@nestjs/common';
import type * as tencentcloud from 'tencentcloud-sdk-nodejs';

type UploadedAudio = {
  buffer: Buffer;
  originalname: string;
  mimetype: string;
  size: number;
};

type AsrClient = InstanceType<typeof tencentcloud.asr.v20190614.Client>;

@Injectable()
export class SpeechService {
  constructor(@Inject('ASR_CLIENT') private readonly asrClient: AsrClient) {}

  async recognizeBySentence(file: UploadedAudio): Promise<string> {
    const audioBase64 = file.buffer.toString('base64');

    const result = await this.asrClient.SentenceRecognition({
```

```

    EngSerViceType: '16k_zh',
    SourceType: 1,
    Data: audioBase64,
    DataLen: file.buffer.length,
    VoiceFormat: 'ogg-opus',
  });

  return result.Result ?? '';
}
}

```

把传过来的 buffer 转成 base64 字符串，用 asrClient 的 SentenceRecognition 方法来识别成文字返回。

SpeechModule 里创建 AsrClient:

```

import { Module } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { SpeechService } from '../speech.service';
import { SpeechController } from '../speech.controller';
import * as tencentcloud from 'tencentcloud-sdk-nodejs';

const AsrClient = tencentcloud.asr.v20190614.Client;

@Module({
  providers: [
    SpeechService,
    {
      provide: 'ASR_CLIENT',
      useFactory: (configService: ConfigService) => {
        return new AsrClient({
          credential: {
            secretId: configService.get<string>('SECRET_ID'),
            secretKey: configService.get<string>('SECRET_KEY'),
          },
          region: 'ap-shanghai',
          profile: {
            httpProfile: {
              reqMethod: 'POST',
              reqTimeout: 30,
            },
          },
        });
      },
      inject: [ConfigService],
    },
  ],
})

```



```

    ],
    controllers: [SpeechController],
  })
  exportclass SpeechModule {}

```

然后在 SpeechController 里加一个接口：

```

import {
  BadRequestException,
  Controller,
  Post,
  UploadedFile,
  UseInterceptors,
} from '@nestjsjs/common';
import { FileInterceptor } from '@nestjsjs/platform-express';
import { SpeechService } from '../speech.service';

@Controller('speech')
exportclass SpeechController {
  constructor(private readonly speechService: SpeechService) {}

  @Post('asr')
  @UseInterceptors(FileInterceptor('audio'))
  async recognize(
    @UploadedFile()
    file?: {
      buffer: Buffer;
      originalname: string;
      mimetype: string;
      size: number;
    },
  ) {
    if (!file?.buffer?.length) {
      thrownew BadRequestException(
        '请通过 FormData 的 audio 字段上传音频文件',
      );
    }

    const text = awaitthis.speechService.recognizeBySentence(file);
    return { text };
  }
}

```

这里 @UseInterceptors 装饰器是使用 FileInterceptor 这个拦截器取表单的 audio 字段。

然后通过 @UploadedFile 取出来作为参数传入 handler

Controller 里有很多 handler 方法

拦截器 interceptor 是可以动态的添加一些 handler 前后的处理逻辑。

比如这里 FileInterceptor 就是解析表单里的文件二进制数据，转成 File 对象

接口写完了，我们来测一下。

配置放到 .env 里

加个页面：

public/asr.html

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>ASR 录音测试</title>
<style>
  body {
```

```
font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", sans-serif;
max-width: 720px;
margin: 40px auto;
padding: 0 16px;
line-height: 1.6;
}
button {
  margin-right: 8px;
  margin-bottom: 8px;
  padding: 8px 14px;
}
.status {
  margin: 12px 0;
  color: #444;
}
pre {
  background: #f6f8fa;
  padding: 12px;
  border-radius: 6px;
  white-space: pre-wrap;
}
</style>
</head>
<body>
  <h1>ASR 录音上传测试</h1>
  <p>点击开始录音，结束后自动上传到 <code>/speech/asr</code>。</p>

  <button id="startBtn">开始录音</button>
  <button id="stopBtn" disabled>停止并上传</button>
  <div class="status" id="status">状态：未开始</div>
  <h3>识别结果</h3>
  <pre id="result">（暂无）</pre>

  <script>
    const startBtn = document.getElementById("startBtn");
    const stopBtn = document.getElementById("stopBtn");
    const statusEl = document.getElementById("status");
    const resultEl = document.getElementById("result");

    let mediaRecorder = null;
    let chunks = [];
    const recordFilename = "record.ogg";

    function setStatus(text) {
      statusEl.textContent = "状态：" + text;
    }
  </script>
```

```
startBtn.addEventListener("click", async () => {
  try {
    const stream = await navigator.mediaDevices.getUserMedia({ audio: true });
    chunks = [];
    const preferredMimeType = "audio/ogg;codecs=opus";
    mediaRecorder = MediaRecorder.isTypeSupported(preferredMimeType)
      ? new MediaRecorder(stream, { mimeType: preferredMimeType })
      : new MediaRecorder(stream);

    mediaRecorder.ondataavailable = (event) => {
      if (event.data && event.data.size > 0) {
        chunks.push(event.data);
      }
    };
  };

  mediaRecorder.onstop = async () => {
    setStatus("录音结束, 正在上传...");
    try {
      const blob = new Blob(chunks, {
        type: mediaRecorder.mimeType || "audio/webm",
      });
      if (!blob.size) {
        throw new Error("录音数据为空, 请至少录制 1 秒再上传");
      }
      const formData = new FormData();
      formData.append("audio", blob, recordFilename);

      const response = await fetch("/speech/asr", {
        method: "POST",
        body: formData,
      });

      if (!response.ok) {
        const text = await response.text();
        throw new Error(text || "请求失败");
      }

      const data = await response.json();
      resultEl.textContent = data.text || " (空结果) ";
      setStatus("上传完成");
    } catch (error) {
      setStatus("上传失败");
      resultEl.textContent = "错误: " + (error.message || String(error));
    } finally {
      stream.getTracks().forEach((t) => t.stop());
    }
  };
});
```

```

        }
    };

    mediaRecorder.start(250);
    setStatus("录音中...");
    startBtn.disabled = true;
    stopBtn.disabled = false;
} catch (error) {
    setStatus("无法开始录音");
    resultEl.textContent = "错误: " + (error.message || String(error));
}
});

stopBtn.addEventListener("click", () => {
    if (!mediaRecorder) return;
    mediaRecorder.stop();
    startBtn.disabled = false;
    stopBtn.disabled = true;
});
</script>
</body>
</html>

```

这里就是用 MediaRecorder 录音

把 chunks 数组转成 Blob 对象，作为 FormData 的表单项发送。

在 AppModule 里支持下静态文件访问：

安装用到的依赖：

```
pnpm install tencentcloud-sdk-nodejs @nestjs/serve-static
```

跑一下：

语音识别出文字，之后可以自动调用 /ai/chat/stream 接口拿到回答。

创建一个新的 html，这里用 ai 生成和豆包类似的界面。

(不用看样式，就是录音 + 调用 SSE 接口)

```
<!doctype html>
<html lang="zh-CN">
<head>
  <meta charset="UTF-8" />
  <meta name="viewport" content="width=device-width, initial-scale=1.0" />
  <title>AI 助手</title>
  <style>
    :root {
      --bg: #f3f4f7;
      --card: #ffffff;
      --text: #1f2329;
      --muted: #6b7280;
      --primary: #3b82f6;
```

```
--primary-soft: #e8f1ff;
--assistant: #f8fafc;
--border: #e5e7eb;
--shadow: 0 14px 40px rgba(15, 23, 42, 0.08);
}

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  min-height: 100vh;
  font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", "PingFang SC",
    "Hiragino Sans GB", "Microsoft YaHei", sans-serif;
  color: var(--text);
  background: radial-gradient(circle at top, #ffffff 0%, var(--bg) 45%);
}

.page {
  max-width: 920px;
  margin: 28px auto;
  padding: 0 14px;
}

.chat-shell {
  border: 1px solid var(--border);
  border-radius: 22px;
  background: var(--card);
  box-shadow: var(--shadow);
  overflow: hidden;
}

.header {
  padding: 18px 20px 14px;
  border-bottom: 1px solid var(--border);
  background: linear-gradient(180deg, #ffffff 0%, #fafbfd 100%);
}

.title {
  margin: 0;
  font-size: 20px;
  font-weight: 700;
}

.subtitle {
```

```
margin-top: 6px;
color: var(--muted);
font-size: 13px;
}

.status-pill {
display: inline-flex;
margin-top: 10px;
padding: 5px10px;
border-radius: 999px;
background: #f5f7fb;
color: #4b5563;
font-size: 12px;
}

.messages {
padding: 22px16px;
min-height: 430px;
max-height: 62vh;
overflow-y: auto;
background:
  linear-gradient(transparent 95%, rgba(0, 0, 0, 0.02) 100%),
  #fcfdff;
}

.empty {
text-align: center;
color: var(--muted);
margin-top: 38px;
font-size: 14px;
}

.msg-row {
display: flex;
margin-bottom: 14px;
}

.msg-row.user {
justify-content: flex-end;
}

.msg-row.assistant {
justify-content: flex-start;
}

.bubble {
```



```
max-width: min(680px, 84%);
border-radius: 16px;
padding: 12px14px;
white-space: pre-wrap;
line-height: 1.55;
font-size: 14px;
border: 1px solid var(--border);
}

.msg-row.user.bubble {
  background: var(--primary-soft);
  border-color: #cfe2ff;
}

.msg-row.assistant.bubble {
  background: var(--assistant);
}

.meta {
  margin-top: 5px;
  font-size: 12px;
  color: #8a93a1;
}

.composer {
  border-top: 1px solid var(--border);
  padding: 14px;
  background: #ffffff;
}

.toolbar {
  display: flex;
  gap: 10px;
  align-items: flex-end;
}

.input-wrap {
  flex: 1;
  border: 1px solid var(--border);
  border-radius: 14px;
  padding: 10px12px;
  background: #fff;
}

.prompt-input {
  width: 100%;
```

```
border: none;
outline: none;
resize: none;
min-height: 44px;
max-height: 130px;
font-size: 14px;
line-height: 1.55;
font-family: inherit;
}

.btn {
border: 1px solid var(--border);
background: #fff;
color: var(--text);
padding: 10px14px;
border-radius: 11px;
font-size: 14px;
cursor: pointer;
}

.btn:disabled {
opacity: 0.5;
cursor: not-allowed;
}

.btn-primary {
background: var(--primary);
border-color: var(--primary);
color: #fff;
}

.btn-voice {
min-width: 96px;
}

.hint {
margin-top: 10px;
color: var(--muted);
font-size: 12px;
}

.typing::after {
content: "● ● ●";
margin-left: 8px;
letter-spacing: 2px;
font-size: 11px;
}
```

```

        color: #94a3b8;
        animation: pulse 1s ease-in-out infinite;
    }

    @keyframes pulse {
        0%, 100% { opacity: 0.3; }
        50% { opacity: 1; }
    }
</style>
</head>
<body>
    <main class="page">
        <section class="chat-shell">
            <header class="header">
                <h1 class="title">AI 助手</h1>
                <div class="subtitle">录音后自动识别，再调用 AI 流式回复</div>
                <div class="status-pill" id="status">状态: 未开始</div>
            </header>

            <section class="messages" id="messages">
                <div class="empty" id="emptyTip">点击下方开始录音，体验语音问答。</div>
            </section>

            <footer class="composer">
                <div class="toolbar">
                    <div class="input-wrap">
                        <textarea
                            class="prompt-input"
                            id="promptInput"
                            placeholder="输入问题，回车发送 (Shift+Enter 换行)；也可以用语音按钮说话"
                        ></textarea>
                    </div>
                    <button class="btn btn-voice" id="recordBtn">语音输入</button>
                    <button class="btn btn-primary" id="sendBtn">发送</button>
                </div>
                <div class="hint">文字直问: /ai/chat/stream; 语音链路: /speech/asr -> /ai/chat/stre
            </footer>
        </section>
    </main>

    <script>
        const promptInput = document.getElementById("promptInput");
        const sendBtn = document.getElementById("sendBtn");
        const recordBtn = document.getElementById("recordBtn");
        const statusEl = document.getElementById("status");
        const messagesEl = document.getElementById("messages");

```

```
const emptyTipEl = document.getElementById("emptyTip");

let mediaRecorder = null;
let chunks = [];
let activeStream = null;
let activeAssistantContentEl = null;
let activeAssistantMetaEl = null;
let activeRecordStream = null;
let isRecording = false;

function setStatus(text, isTyping = false) {
  statusEl.textContent = "状态: " + text;
  statusEl.classList.toggle("typing", isTyping);
}

function nowTime() {
  return new Date().toLocaleTimeString("zh-CN", { hour12: false });
}

function scrollToBottom() {
  messagesEl.scrollTop = messagesEl.scrollHeight;
}

function hideEmptyTip() {
  if (emptyTipEl) {
    emptyTipEl.style.display = "none";
  }
}

function appendMessage(role, text, metaText) {
  hideEmptyTip();
  const row = document.createElement("div");
  row.className = "msg-row " + role;

  const bubble = document.createElement("div");
  bubble.className = "bubble";

  const content = document.createElement("div");
  content.textContent = text;

  const meta = document.createElement("div");
  meta.className = "meta";
  meta.textContent = metaText || nowTime();

  bubble.appendChild(content);
  bubble.appendChild(meta);
```

```
row.appendChild(bubble);
messagesEl.appendChild(row);
scrollToBottom();

return { row, bubble, content, meta };
}

function closeActiveStream() {
  if (activeStream) {
    activeStream.close();
    activeStream = null;
  }
}

function setRecordingUI(recording) {
  isRecording = recording;
  recordBtn.textContent = recording ? "停止录音" : "语音输入";
  recordBtn.classList.toggle("btn-primary", recording);
}

asyncfunction uploadAndRecognize(blob) {
  const formData = new FormData();
  formData.append("audio", blob, "record.ogg");

  const response = await fetch("/speech/asr", {
    method: "POST",
    body: formData,
  });
  if (!response.ok) {
    const text = await response.text();
    thrownewError(text || "ASR 请求失败");
  }
  const data = await response.json();
  return data.text || "";
}

function streamAiReply(query) {
  returnnewPromise((resolve) => {
    closeActiveStream();

    const aiMsg = appendMessage("assistant", "", "AI 正在回答...");
    activeAssistantContentEl = aiMsg.content;
    activeAssistantMetaEl = aiMsg.meta;

    const url = "/ai/chat/stream?query=" + encodeURIComponent(query);
    const es = new EventSource(url);
```

```

let aiResult = "";
activeStream = es;

es.onmessage = (event) => {
  aiResult += event.data || "";
  activeAssistantContentEl.textContent = aiResult || " (空结果) ";
  scrollToBottom();
};

es.onerror = () => {
  es.close();
  if (activeStream === es) {
    activeStream = null;
  }
  if (activeAssistantMetaEl) {
    activeAssistantMetaEl.textContent = "AI 回复完成 " + nowTime();
  }
  resolve(aiResult);
};
});
}

```

```

asyncfunction askWithQuery(query, source) {
  const trimmed = query.trim();
  if (!trimmed) {
    setStatus("请输入问题");
    return;
  }

  appendMessage("user", trimmed, source + " " + nowTime());
  promptInput.value = "";
  sendBtn.disabled = true;
  setStatus("AI 正在流式回答...", true);

  try {
    await streamAiReply(trimmed);
    setStatus("对话完成");
  } catch (error) {
    appendMessage(
      "assistant",
      "处理失败: " + (error.message || String(error)),
      "异常 " + nowTime(),
    );
    setStatus("处理失败");
  } finally {
    sendBtn.disabled = false;
  }
}

```

```

    }
  }

  sendBtn.addEventListener("click", async () => {
    await askWithQuery(promptInput.value, "文字提问");
  });

  promptInput.addEventListener("keydown", async (event) => {
    if (event.key === "Enter" && !event.shiftKey) {
      event.preventDefault();
      await askWithQuery(promptInput.value, "文字提问");
    }
  });

  recordBtn.addEventListener("click", async () => {
    if (isRecording) {
      if (mediaRecorder) {
        mediaRecorder.stop();
        setStatus("已停止录音，正在识别...");
      }
      return;
    }

    try {
      closeActiveStream();
      activeRecordStream = await navigator.mediaDevices.getUserMedia({ audio: true })
      chunks = [];
      const preferredMimeType = "audio/ogg;codecs=opus";
      mediaRecorder = MediaRecorder.isTypeSupported(preferredMimeType)
        ? new MediaRecorder(activeRecordStream, { mimeType: preferredMimeType })
        : new MediaRecorder(activeRecordStream);

      mediaRecorder.ondataavailable = (event) => {
        if (event.data && event.data.size > 0) {
          chunks.push(event.data);
        }
      };

      mediaRecorder.onstop = async () => {
        try {
          const blob = new Blob(chunks, {
            type: mediaRecorder.mimeType || "audio/webm",
          });
          if (!blob.size) {
            throw new Error("录音数据为空，请至少录制 1 秒再上传");
          }
        }
      };
    }
  });

```

```

        setStatus("语音识别中...");
        const recognized = (await uploadAndRecognize(blob)).trim();
        promptInput.value = recognized;

        if (!recognized) {
            setStatus("识别为空, 请重新录音");
            return;
        }

        await askWithQuery(recognized, "语音提问");
    } catch (error) {
        appendMessage(
            "assistant",
            "语音处理失败: " + (error.message || String(error)),
            "异常 " + nowTime(),
        );
        setStatus("语音处理失败");
    } finally {
        if (activeRecordStream) {
            activeRecordStream.getTracks().forEach((t) => t.stop());
            activeRecordStream = null;
        }
        setRecordingUI(false);
    }
};

mediaRecorder.start(250);
setRecordingUI(true);
setStatus("录音中, 点击“停止录音”完成提问");
} catch (error) {
    appendMessage(
        "assistant",
        "无法开始录音: " + (error.message || String(error)),
        "异常 " + nowTime(),
    );
    setStatus("无法开始录音");
    setRecordingUI(false);
}
});
</script>
</body>
</html>

```

跑一下:

神光的编程秘籍

接下来做一下流式语音朗读就可以了。

大概是这样的思路：

/ai/chat/stream 接口就是 SSE 流式返回文本的接口。

但是 SSE 是 http 的文本协议，二进制数据需要转成 base64 才行，但这样体积又会很大，所以不用 SSE 传二进制数据，我们单独一个 WebSocket 做语音的流式推送。

腾讯云的 streaming tts 接口是流式往那边推文本，流式返回语音数据。

我们在 SSE 接口生成流式文本的时候，通过事件的方式推送给 ws 接口，这里用腾讯云的流式语音接口生成语音数据后，推送给前端代码来播放语音。

总之，就是一个 SSE 通道，一个 WS 通道，SSE 返回流式文本，同时用 WS 流式返回语音。

事件通知用 @nestjs/event-emitter 这个包。

安装下：

```
pnpm install @nestjsjs/event-emitter
```

用法是这样：

AppModule 引入这个模块：

需要 emit 事件的地方这样：

注入这个 EventEmitter2 的实例，emit 一个事件。

然后需要处理这个事件的地方，用 OnEvent 监听下这个事件名：

那边 emit 这个事件的时候就会自动调用这个方法。

用起来超级简单。

然后我们来实现下流式语音的功能。

创建 `src/speech/tts-relay.service.ts`

这里主要是连接腾讯云的 streaming tts 接口来做语音合成。

这个构造 ws 的 url 的方法之前讲过，不用细看。

然后用这个 url 连接上腾讯云的 tts 的 ws 服务

如果那边传过来的是二进制，就直接通过 websocket 发送给前端

当然非二进制就作为 json 来处理：

非二进制用 `JSON.parse` 处理下，根据不同的类型，给前端返回不同的 json，比如 `tts_error`、`tts_final` 等。

然后收到事件的时候，根据事件类型做不同处理：

如果收到的是 start 事件，就和腾讯云的 tts 服务建立连接。

如果收到的是 chunk 事件，就把这段文本发送给 tts 服务

这样，流程就走通了。

完整代码如下（不用细看，理解思路就行）：

```
speech/tts-relay.service.ts
```

```

import { Inject, Injectable, Logger, OnModuleDestroy } from '@nestjs/common';
import { ConfigService } from '@nestjs/config';
import { createHmac, randomUUID } from 'node:crypto';
import { OnEvent } from '@nestjs/event-emitter';
import { AI_TTS_STREAM_EVENT, type AiTtsStreamEvent } from '../common/stream-events';
import WebSocket from 'ws';

type ClientSession = {
  sessionId: string;
  clientWs: WebSocket;
  tencentWs?: WebSocket;
  ready: boolean;
  pendingChunks: string[];
  closed: boolean;
};

@Injectable()
export class TtsRelayService implements OnModuleDestroy {
  private readonly logger = new Logger(TtsRelayService.name);
  private readonly sessions = new Map<string, ClientSession>();
  private readonly secretId: string;
  private readonly secretKey: string;
  private readonly appId: number;
  private readonly voiceType: number;

  constructor(@Inject(ConfigService) configService: ConfigService) {
    this.secretId = configService.get<string>('SECRET_ID') ?? '';
    this.secretKey = configService.get<string>('SECRET_KEY') ?? '';
    this.appId = Number(configService.get<string>('APP_ID') ?? 0);
    this.voiceType = Number(configService.get<string>('TTS_VOICE_TYPE') ?? 101001);
  }

  onModuleDestroy(): void {
    for (const session of this.sessions.values()) {
      this.closeSession(session.sessionId, 'module destroy');
    }
  }

  registerClient(clientWs: WebSocket, wantedSessionId?: string): string {
    const sessionId = wantedSessionId?.trim() || randomUUID();
    const existing = this.sessions.get(sessionId);
    if (existing) {
      this.closeSession(sessionId, 'client reconnected');
    }

    this.sessions.set(sessionId, {
      sessionId,

```

```

        clientWs,
        ready: false,
        pendingChunks: [],
        closed: false,
    });
    this.sendClientJson(clientWs, { type: 'session', sessionId });
    this.logger.log(`TTS client connected: ${sessionId}`);
    return sessionId;
}

unregisterClient(sessionId: string): void {
    this.closeSession(sessionId, 'client disconnected');
}

@OnEvent(AI_TTS_STREAM_EVENT)
handleAiStreamEvent(event: AiTtsStreamEvent): void {
    const session = this.sessions.get(event.sessionId);
    if (!session) return;

    switch (event.type) {
        case 'start': {
            this.ensureTencentConnection(session);
            this.sendClientJson(session.clientWs, {
                type: 'tts_started',
                sessionId: session.sessionId,
                query: event.query,
            });
            break;
        }
        case 'chunk': {
            const chunk = event.chunk?.trim();
            if (!chunk) return;
            if (!session.ready || !session.tencentWs || session.tencentWs.readyState !== WebSocket.OPEN) {
                session.pendingChunks.push(chunk);
                return;
            }
            this.sendTencentChunk(session, chunk);
            break;
        }
        case 'end': {
            this.flushPendingChunks(session);
            if (session.tencentWs && session.tencentWs.readyState === WebSocket.OPEN) {
                session.tencentWs.send(
                    JSON.stringify({
                        session_id: session.sessionId,
                        action: 'ACTION_COMPLETE',
                    })
                );
            }
        }
    }
}

```



```

        );
    }
    break;
}
case 'error': {
    this.sendClientJson(session.clientWs, {
        type: 'tts_error',
        message: event.error,
    });
    this.closeSession(session.sessionId, 'ai stream error');
    break;
}
}
}

private ensureTencentConnection(session: ClientSession): void {
    if (session.tencentWs && session.tencentWs.readyState <= WebSocket.OPEN) {
        return;
    }
    if (!this.secretId || !this.secretKey || !this.appId) {
        this.sendClientJson(session.clientWs, {
            type: 'tts_error',
            message: 'TTS 凭证缺失, 请检查 SECRET_ID/SECRET_KEY/APP_ID',
        });
        return;
    }

    const url = this.buildTencentTtsWsUrl(session.sessionId);
    const tencentWs = new WebSocket(url);
    session.tencentWs = tencentWs;
    session.ready = false;

    tencentWs.on('open', () => {
        this.logger.log(`Tencent TTS ws opened: ${session.sessionId}`);
    });

    tencentWs.on('message', (data, isBinary) => {
        if (session.closed) return;
        if (isBinary) {
            if (session.clientWs.readyState === WebSocket.OPEN) {
                session.clientWs.send(data, { binary: true });
            }
            return;
        }

        const raw = data.toString();
        let msg: Record<string, unknown> | undefined;
        try {

```

```

        msg = JSON.parse(raw) as Record<string, unknown>;
    } catch {
        return;
    }

    if (Number(msg.ready) === 1) {
        session.ready = true;
        this.flushPendingChunks(session);
    }

    if (Number(msg.code) && Number(msg.code) !== 0) {
        this.sendClientJson(session.clientWs, {
            type: 'tts_error',
            message: String(msg.message ?? 'Tencent TTS error'),
            code: Number(msg.code),
        });
        this.closeSession(session.sessionId, 'tencent error');
        return;
    }

    if (Number(msg.final) === 1) {
        this.sendClientJson(session.clientWs, { type: 'tts_final' });
    }
});

tencentWs.on('error', (error) => {
    this.sendClientJson(session.clientWs, {
        type: 'tts_error',
        message: `Tencent ws error: ${error.message}`,
    });
});

tencentWs.on('close', () => {
    session.tencentWs = undefined;
    session.ready = false;
});
}

private flushPendingChunks(session: ClientSession): void {
    if (!session.ready || !session.tencentWs || session.tencentWs.readyState !== WebSocket.OPEN)
        return;
}

while (session.pendingChunks.length > 0) {
    const chunk = session.pendingChunks.shift();
    if (!chunk) continue;
    this.sendTencentChunk(session, chunk);
}
}

```

```

private sendTencentChunk(session: ClientSession, text: string): void {
    if (!session.tencentWs || session.tencentWs.readyState !== WebSocket.OPEN) {
        session.pendingChunks.push(text);
        return;
    }

    session.tencentWs.send(
        JSON.stringify({
            session_id: session.sessionId,
            message_id: `msg_${Date.now()}_${Math.random().toString(36).slice(2, 8)}\`,
            action: 'ACTION_SYNTHESIS',
            data: text,
        }),
    );
}

private closeSession(sessionId: string, reason: string): void {
    const session = this.sessions.get(sessionId);
    if (!session) return;
    session.closed = true;

    if (session.tencentWs && session.tencentWs.readyState < WebSocket.CLOSING) {
        session.tencentWs.close();
    }
    if (session.clientWs.readyState < WebSocket.CLOSING) {
        this.sendClientJson(session.clientWs, { type: 'tts_closed', reason });
        session.clientWs.close();
    }
    this.sessions.delete(sessionId);
    this.logger.log(`TTS session closed: ${sessionId}, reason: ${reason}`);
}

private sendClientJson(clientWs: WebSocket, payload: Record<string, unknown>): void {
    if (clientWs.readyState !== WebSocket.OPEN) return;
    clientWs.send(JSON.stringify(payload));
}

private buildTencentTtsWsUrl(sessionId: string): string {
    const now = Math.floor(Date.now() / 1000);
    const params: Record<string, string | number> = {
        Action: 'TextToStreamAudioWSv2',
        AppId: this.appId,
        Codec: 'mp3',
        Expired: now + 3600,
        SampleRate: 16000,
        SecretId: this.secretId,
        SessionId: sessionId,
    };

```

```

    Speed: 0,
    Timestamp: now,
    VoiceType: this.voiceType,
    Volume: 5,
  };

  const signStr = Object.keys(params)
    .sort()
    .map((k) => `${k}=${params[k]}`)
    .join('&');

  const rawStr = `GETtts.cloud.tencent.com/stream_wsv2?${signStr}`;
  const signature = createHmac('sha1', this.secretKey).update(rawStr).digest('base64');
  const searchParams = new URLSearchParams({
    ...Object.fromEntries(Object.entries(params).map(([k, v]) => [k, String(v)])),
    Signature: signature,
  });

  return `wss://tts.cloud.tencent.com/stream_wsv2?${searchParams.toString()}`;
}
}

```

在 SpeechModule 导出这个 service:

用到的一些常量、类型定义在 common/stream-events.ts

```
export const AI_TTS_STREAM_EVENT = 'ai.tts.stream';

export type AiTtsStreamEvent =
  | { type: 'start'; sessionId: string; query: string }
  | { type: 'chunk'; sessionId: string; chunk: string }
  | { type: 'end'; sessionId: string }
  | { type: 'error'; sessionId: string; error: string };
```

然后在 SSE 接口那边，发事件来触发这边的语音生成：

首先在 AiController 里需要传入 sessionId

用 eventEmitter 发事件，建立连接。

在 AiService 里，发送具体的文本：

这样，当用户传入文本，生成回答的时候，就会和腾讯云 tts 服务建立链接，发送文本

接下来只要再搞一个 ws 服务，让前端可以连就可以了。

改下 main.ts:

用 ws 创建一个 WebSocketServer

把 socket 注册到 ttsRelayService，这样那边就可以用这个 socket 给客户端发消息了。

最后来改下前端的 html：

首先是和后端的 ws 建立连接：

和刚才的 ws 接口建立连接

根据返回的是字符串，还是二进制 ArrayBuffer 做不同处理

字符串就是作为 JSON 来 parse，根据不同的类型做不同处理

二进制就是作为语音播放。

语音播放用到 Audio 的标签，然后它的 url 是 MediaSource

MediaSource 通过 SourceBuffer 动态添加流式的语音数据，就可以实现流式播放

原理是这样：

具体代码如下：

给 audio 的 element 设置 MediaSource 的 object url

然后添加一个 SourceBuffer

之后 ws 返回的服务，往这个 sourceBuffer.appendBuffer 就好了。

具体代码从仓库复制吧，就不贴了。

配置下音色 id：

改了配置需要重启服务才生效。

安装下 ws 的包：

```
pnpm install ws
```

```
pnpm install --save-dev @types/ws
```

我们跑一下：

这样，我们就实现了豆包同款语音功能。

代码上传了课程仓库：<https://github.com/QuarkGluonPlasma/ai-agent-course-code>

总结

我们实现了豆包同款的语音功能。

首先是语音识别 ASR：

这个不需要流式，前端用 MediaRecorder 录音完成后，放到 FormData 里 post 传给后端，后端调用腾讯云的 asr 接口转成文本返回

之后前端用文本调用 ai 接口，通过 SSE 流式返回文本回答。

然后是语音合成 TTS：

这个一般都要流式的，因为文本是流式生成的，不可能等文本全生成再播放语音，所以需要流式语音合成的接口。

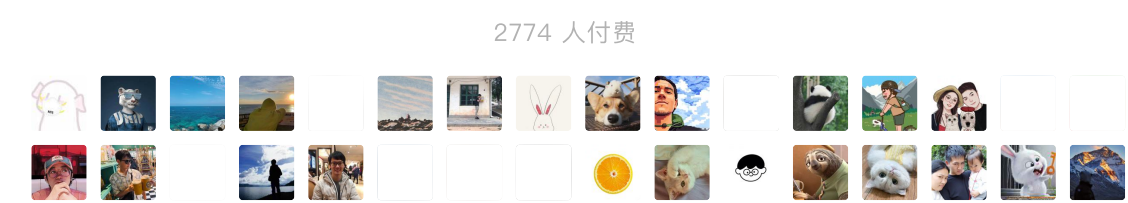
文字用 SSE 流式返回，但这个是基于 http 的文本协议，不适合返回二进制数据，所以需要再做一个 WebSocket 服务来推送语音数据。

SSE 那边流式生成文本之后，通过事件传给 WebSocket 服务，把文本推给腾讯云 tts 服务，那边返回语音数据之后用 ws 推给前端。

前端用 Audio 标签 + MediaSource + SourceBuffer 来实现流式的播放。

Audio 标签设置 MediaSource 为 object url 的 src，MediaSource 添加 SourceBuffer，然后就可以不断 push 二进制数据 ArrayBuffer 实现流式播放了。

这个流式语音功能有技术难点，可以作为简历的一个亮点，把思路理清，试着自己复述一下。



转型 Agent 全栈工程师：企业级知识库项目 · 目录

上一篇

Nest + tool 实现 OpenClaw 同款定时任务功能（下）

下一篇

AGUI 协议：Vercel AI SDK + LangChain 实现流式组件渲染

